

COMPUTER NETWORK PROJECT PROPOSAL

ข้อเสนอการพัฒนาโปรแกรม Network Application และการออกแบบ Application-Layer Protocol (SRMP)

วิชาเครือข่ายคอมพิวเตอร์ (Computer Networks) — รายงานการออกแบบโปรโตคอลและสถาปัตยกรรมระบบ

1. รายละเอียดและการเสนอการพัฒนาโปรแกรม Network Application

1.1 วัตถุประสงค์ของโปรแกรม (Program Objective & Description)

โปรแกรมที่พัฒนาขึ้นมีชื่อว่า "SRMP Monitor (System Resource Monitoring Protocol Monitor)" เป็น Network Application ประเภท Real-Time Remote System Resource Monitoring & Management System ที่พัฒนาขึ้นตามสถาปัตยกรรมแบบ Client-Server เพื่อตอบสนองการบริหารจัดการและตรวจสอบสถานะคอมพิวเตอร์แม่ข่ายหรือเครื่องปลายทาง (Host PC) จากระยะไกลผ่านระบบเครือข่าย Local Area Network (LAN / Wi-Fi)

วัตถุประสงค์และขอบเขตการทำงานหลักของโปรแกรม:

- การติดตามสถานะทรัพยากรฮาร์ดแวร์แบบ Real-Time (Telemetry):** ดึงข้อมูลสถานะการทำงานระดับลึก ได้แก่ อัตราการใช้งาน CPU (%), RAM (%), Disk Storage (%), อุณหภูมิหน่วยประมวลผล (CPU Package Temp, GPU Temp), ความเร็วเครือข่าย Upload/Download (MB/s), สถานะการเชื่อมต่ออินเทอร์เน็ตพร้อมค่า Ping Latency และระยะเวลาเปิดเครื่อง (System Uptime)
- การบริหารจัดการโปรเซส (Process Management):** ตรวจสอบรายการโปรเซสที่ใช้ทรัพยากรสูงสุด (Top Processes) เรียงลำดับตาม CPU หรือ RAM และสามารถสั่งยุติการทำงานของโปรเซสที่ไม่พึงประสงค์ได้ทันที (Remote Process Termination / Kill Task)
- การควบคุมระบบและฮาร์ดแวร์จากระยะไกล (Remote Hardware & System Control):** ปรับระดับความดังของเสียง (Volume Control 0-100%), ปรับระดับความสว่างของหน้าจอ (Screen Brightness 0-100%), ล็อกหน้าจอ (Lock Screen), รีบูต (Restart) และสั่งปิดเครื่องคอมพิวเตอร์ (Shutdown) พร้อมระบบนับถอยหลัง ป้องกันความผิดพลาด
- การเปิดเครื่องระยะไกลผ่านเครือข่าย (Wake-on-LAN: WOL):** รองรับการส่งสัญญาณ Magic Packet เพื่อเปิดเครื่องคอมพิวเตอร์ที่ปิดอยู่ (Power Off) ให้พร้อมทำงานจากระยะไกล
- ระบบแจ้งเตือนเหตุฉุกเฉิน (Push Alert Notifications):** การแจ้งเตือนทันทีเมื่อตรวจพบค่าทรัพยากรของเครื่องทำงานหนักเกินเกณฑ์วิกฤต (เช่น อุณหภูมิสูงเกิน หรือ RAM เต็ม)

1.2 คุณลักษณะของ Application ที่พัฒนา (Application Characteristics)

- สถาปัตยกรรมการสื่อสารแบบผสมผสาน (Dual-Layer Client-Server Model):**
 - ฝั่ง Server (Python Backend):** ทำหน้าที่เป็น Service Daemon คอยเก็บข้อมูลจากระดับ Operating System (OS APIs, WMI, LibreHardwareMonitor DLL, psutil, nvidia-smi) และคอยรับคำสั่งควบคุมระบบ
 - ฝั่ง Client (Cross-Platform Flutter App):** ทำหน้าที่เป็นหน้าจอควบคุม (Interactive GUI Dashboard) รองรับทั้งสมาร์ทโฟน (Android/iOS) และคอมพิวเตอร์ พร้อมรองรับระบบ Mobile Home Widget เพื่อแสดงผลสถิติบนหน้าจอหลักของโทรศัพท์

- **Real-Time Data Streaming & Low Latency:** ส่งข้อมูลสถิติของเครื่องอย่างต่อเนื่องทุก 1 วินาที มีความหน่วงต่ำ เพื่อให้ผู้ดูแลระบบมองเห็นสถานะปัจจุบันของเครื่องเสมือนนั่งอยู่หน้าเครื่องจริง
- **การสื่อสารแบบสองทิศทาง (Interactive & Bidirectional Control):** มีทั้งระบบ Push สตรีมข้อมูลจาก Server สู Client, ระบบ Request-Response ตอบสนองคำสั่งทันที และระบบ Server-Push Alert แจ้งเตือนข้อผิดพลาด
- **ความทนทานต่อข้อผิดพลาด (Fault Tolerance & Non-blocking I/O):** ระบบทำงานแบบ Asynchronous I/O และ Multi-threading หากข้อมูลสถิติบางช่วงเวลาสูญหาย หน้าจอ UI จะไม่หยุดทำงานและทำการดึงข้อมูลใหม่ทันที

2. การเลือกใช้ Service Model ของ Transport Layer (UDP vs TCP)

ในการพัฒนาระบบ SRMP ได้เลือกใช้ สถาปัตยกรรมแบบไฮบริด (Hybrid Transport Layer Architecture) โดยแบ่งการทำงานออกเป็น 2 ท่อสัญญาณตามลักษณะความต้องการของข้อมูล (Application Requirements) ดังนี้:

UDP (Port 9000) 1. การเลือกใช้ UDP สำหรับ Metric Telemetry Broadcast และ Wake-on-LAN

เหตุผลที่เลือกใช้ UDP (User Datagram Protocol):

1. **Low Overhead & Minimal Latency (ประหยัดแบนด์วิธและไม่มีควมหน่วงสะสม):** UDP เป็น Connectionless Protocol มี Header ขนาดเล็กเพียง 8 Bytes และไม่ต้องเสียเวลาทำ 3-Way Handshake ก่อนส่งข้อมูล จึงเหมาะกับการส่งสตรีมข้อมูลสถิติ (CPU, RAM, Temp) ที่เกิดขึ้นถี่ๆ ทุก 1 วินาที
2. **ยอมรับการสูญหายของข้อมูลได้ (Tolerate Packet Loss):** ข้อมูลสถิติเป็นข้อมูลประเภท Real-Time Time-Sensitive Data หากแพ็กเก็ตของวินาทีที่ผ่านมาสูญหายไประหว่างทาง *ไม่มีความจำเป็นต้องส่งซ้ำ (No Retransmission)* เพราะข้อมูลใหม่ล่าสุดในวินาทีถัดไปจะถูกส่งมาแทนที่ทันที การส่งซ้ำของ TCP จะทำให้เกิดความหน่วงสะสมที่ไม่พึงประสงค์
3. **ความสามารถในการส่งแบบ Broadcast (1-to-Many Transmission):** UDP รองรับการส่งกระจายสัญญาณแบบ `255.255.255.255` ทำให้ Server ส่งแพ็กเก็ตเพียงครั้งเดียว แต่อุปกรณ์ Client หลากหลายเครื่องในวงเครือข่ายสามารถรับฟังข้อมูลพร้อมกันได้ โดย Server ไม่ต้องสิ้นเปลืองหน่วยความจำในการเปิด Connection ค้างไว้
4. **ความเข้ากันได้กับมาตรฐาน Wake-on-LAN:** การส่ง Magic Packet เพื่อเปิดเครื่องคอมพิวเตอร์ที่ปิดอยู่ ต้องส่งผ่านสัญญาณ UDP Broadcast ไปยัง Port 9 หรือ 7 เท่านั้น เพื่อให้การ์ดแลน (NIC) ตอบรับสัญญาณได้ในระดับฮาร์ดแวร์

TCP (Port 9001) 2. การเลือกใช้ TCP สำหรับ Command & Control Channel และ Alert Notifications

เหตุผลที่เลือกใช้ TCP (Transmission Control Protocol):

1. **Reliability & Guaranteed Delivery (การันตีว่าข้อมูลถึงปลายทางและไม่เสียหาย 100%):** คำสั่งควบคุม เช่น `KILL_PROC`, `SYS_POWER action=SHUTDOWN`, `SET_VOL` เป็นคำสั่งที่มีความสำคัญสูงมาก (Critical Operations) หากข้อมูลตกหล่นจะส่งผลให้เครื่องไม่ปฏิบัติตามคำสั่ง จึงต้องพึ่งพาคุณสมบัติ Acknowledgement (ACK) และ Automatic Retransmission ของ TCP
2. **รักษาลำดับความถูกต้องของคำสั่ง (In-Order Delivery):** TCP รับประกันว่าคำสั่ง Request และข้อความตอบกลับ Response (เช่น `200 OK`, `404 NOT_FOUND`) จะส่งถึงปลายทางตามลำดับอย่างถูกต้องผ่านระบบ Sequence Number

3. **Connection-Oriented & State Management:** การเชื่อมต่อแบบเปิดค้างไว้ (Persistent Socket) ช่วยให้ทั้ง Client และ Server ตรวจสอบสถานะ Online/Offline ของกันและกันได้ทันที และทำให้ Server สามารถส่งข้อความเตือนฉุกเฉิน (202 ALERT) แบบ Server-Push กลับมายัง Client ได้โดยตรง

มิติการเปรียบเทียบ	UDP Channel (Port 9000)	TCP Channel (Port 9001)
ฟังก์ชันการทำงาน	สตรีมข้อมูลสถิติ (Metrics) & Wake-on-LAN	คำสั่งควบคุม (Commands), ผลตอบกลับ (Response), แจ้งเตือน (Alerts)
ลักษณะการเชื่อมต่อ	Connectionless (ส่งทันทีโดยไม่ต้องเชื่อมต่อ)	Connection-Oriented (Three-Way Handshake)
การรับประกันความน่าเชื่อถือ	Unreliable (ไม่การันตี, ไม่มีการส่งซ้ำ)	Reliable (รับประกันข้อมูลถึงครบ 100%, มี Retransmission)
ทิศทางและรูปแบบการส่ง	One-Way Broadcast (1-to-Many)	Two-Way Interactive Unicast (Client ↔ Server)
Overhead & Header Size	ต่ำมาก (Header 8 Bytes)	ปานกลาง (Header 20 Bytes + ควบคุมสถานะท่อเชื่อมต่อ)

3. การออกแบบ Application-Layer Protocol (SRMP: System Resource Monitoring Protocol)

3.1 ภาพรวมของโปรโตคอล (Protocol Overview & Specification)

ชื่อโปรโตคอล: SRMP (System Resource Monitoring Protocol) v1.0

รูปแบบข้อความ (Message Encoding): Plain-text ASCII / UTF-8 String ที่อ่านและ Debug ได้ง่าย (Human-readable & Self-describing) สิ้นสุดข้อความด้วยอักขระขึ้นบรรทัดใหม่ (`\n` หรือ `\r\n`)

3.2 โครงสร้างข้อความ UDP Metric Broadcast (Server → Clients)

Server ทำการ Broadcast ข้อมูลไปยัง Port 9000 ทุกๆ 1 วินาที โดยส่งเป็นข้อความแบบ Plain-Text (Key-Value Pair) ดังนี้:

รูปแบบข้อความ: METRIC cpu=<CPU%> ram=<RAM%> disk=<DISK%> temp_cpu=<TEMP_C> temp_gpu=<TEMP_C> net_up=<MB/s> net_down=<MB/s> net_online=<0|1> net_ping=<MS> uptime=<SECS>

ตัวอย่างข้อความที่ส่งจริง:

METRIC cpu=24.5% ram=68.2% disk=45.0% temp_cpu=58.5C temp_gpu=46.0C net_up=0.12MB/s net_down=1.45MB/s net_online=1 net_ping=18ms uptime=43200s

ความหมายของข้อมูล: CPU ทำงาน 24.5%, RAM 68.2%, Disk 45.0%, ความร้อน CPU 58.5°C, ความร้อน GPU 46.0°C, ความเร็วเน็ต Up 0.12 MB/s, Down 1.45 MB/s, สถานะเน็ตออนไลน์ ปิงเฉลี่ย 18ms, เครื่องเปิดใช้งานมาแล้ว 43,200 วินาที (12 ชั่วโมง)

3.3 โครงสร้างคำสั่งและข้อความตอบกลับ TCP (Client ↔ Server)

การสื่อสารผ่าน TCP Port 9001 ใช้รูปแบบ Request-Response และ Server-Push:

- Request Message Syntax:** <COMMAND_VERB> [arg1=value1] [arg2=value2]...
- Response Message Syntax:** <STATUS_CODE> <STATUS_PHRASE> - <RESPONSE_BODY>

หมวดหมู่	Request Message (Client → Server)	Response Message (Server → Client)
Process List	GET_TOP_PROCS limit=8 sortby=CPU (sortby รองรับ CPU หรือ RAM)	200 OK - procs=[{pid:1240,name:chrome.exe,cpu:15.2%,ram:8.4%,ram_mb:1344.2MB},...]
Kill Process	KILL_PROC pid=1240	200 OK - PROC_KILLED หรือ 404 NOT_FOUND - PROCESS_NOT_EXIST
Get Setting	GET_SETTING name=volume GET_SETTING name=brightness	200 OK - SETTING_VALUE name=volume value=65

หมวดหมู่	Request Message (Client → Server)	Response Message (Server → Client)
Set Volume	SET_VOL level=75	200 OK - VOLUME_SET_75
Set Brightness	SET_BRIGHTNESS level=80	200 OK - BRIGHTNESS_SET_80
System Power	SYS_POWER action=LOCK SYS_POWER action=RESTART SYS_POWER action=SHUTDOWN	200 OK - SYSTEM_LOCKED 200 OK - SYSTEM_RESTARTING 200 OK - SYSTEM_SHUTDOWN
Server-Push Alert	(ส่งอัตโนมัติจาก Server เมื่อเกิดเหตุฉุกเฉิน)	202 ALERT - CPU Temperature exceeded 85C threshold

3.4 รหัสสถานะการทำงาน (SRMP Status Codes)

- **200 OK** : คำสั่งได้รับการประมวลผลและทำงานสำเร็จสมบูรณ์
- **202 ALERT** : ข้อความแจ้งเตือนด่วนแบบ Push Notification จากเซิร์ฟเวอร์
- **400 BAD_REQUEST** : รูปแบบคำสั่งหรือพารามิเตอร์ไม่ถูกต้อง เช่น ค่าระดับเสียงเกิน 100
- **404 NOT_FOUND** : ไม่พบโปรเซสเป้าหมาย (PID) หรือการตั้งค่าที่ร้องขอ
- **500 INTERNAL_ERROR** : เกิดข้อผิดพลาดภายในระบบแม่ข่ายในการเรียกใช้งาน OS API

3.5 แผนผังลำดับขั้นตอนการสื่อสาร (Message Exchange Flow Diagram)

```

[ Client (Flutter App) ]                                [ Server (Python Daemon) ]
|                                                           |
| ----- 1. UDP Broadcast Listener (Port 9000) ----- | (Listening)
| <..... UDP Packet: METRIC cpu=24% ram=68% .....> | (Every 1 sec)
|                                                           |
| ===== 2. TCP Handshake & Connect (Port 9001) ===== |
| -----> | (Connection Established)
|                                                           |
| ----- GET_TOP_PROCS limit=8 sortby=CPU -----> |
| <----- 200 OK - procs=[{pid:1240,name:...}] ----- |
|                                                           |
| ----- SET_VOL level=75 -----> |
| <----- 200 OK - VOLUME_SET_75 ----- |
|                                                           |
| ----- KILL_PROC pid=1240 -----> |
| <----- 200 OK - PROC_KILLED ----- |
|                                                           |
| <----- 202 ALERT - CPU Temp High (88°C) ----- | (Push Event)
|                                                           |

```